WHAT WE CAN ACTUALLY BUILD, WHAT WE SHOULD NOT CLAIM, AND WHAT THIS TEAM CAN FINISH FAST.

Use this as the working packet for the group. It is not the whitepaper and it is not the full repo explanation. It is the shortest honest path from P4P to something the team can align around, divide, build, and demo together.

Core claim: a local takeaway shop can be reachable by QR/link, show a menu, receive a pickup order, and keep the order path out of a marketplace middleman.

Build together: one direct pickup-ordering path for one local takeaway shop.

Say clearly: discovery can be public, but the order should still go directly to the shop.

Ask later: pilot partner, small follow-up support, and practical legal/regulatory help.

AGREE THIS BEFORE WE PITCH

- [ ] We agree on the shortest sentence: **direct pickup ordering for local takeaway shops without marketplace capture**.
- [ ] We agree the hackathon demo is smaller than the full P4P repo.
- [ ] We agree the registry is for discovery, not order relay.
- [ ] We agree this is **not** "Uber Eats but open source."
- [ ] We agree on one clear user: a small local takeaway shop that wants a direct route back to the customer.

WHAT WE CAN FINISH TOGETHER

MINIMUM DEMO

- [ ] one shop page and readable menu
- [ ] QR code or direct link
- [ ] simple pickup-order flow
- [ ] small operator screen
- [ ] **received** and **ready** status
- [ ] payment at pickup first

STRONGER IF TIME ALLOWS

- [ ] one small registry for discovery
- [ ] one small node endpoint for menu and order
- [ ] explicit **order_mode**
- [ ] request trace that proves menu/order hit the node directly
- [ ] backup registry failover for discovery

WHAT SUCCESS LOOKS LIKE

- [ ] A customer opens a QR code or direct link.
- [ ] The customer sees the shop page and menu.
- [ ] The customer places a pickup order.
- [ ] The shop sees the order on its operator screen.
- [ ] The order moves from **received** to **ready**.
- [ ] The team can explain in one sentence why this is different from a platform marketplace.
- [ ] If using the technical variant, the team can show discovery used a registry but the order did not go through the registry.

WHAT WE SAY HONESTLY IN THE ROOM

- This is a proof of direct ordering, not a finished business rollout.
- This is a prototype, not a production restaurant system.
- The value is the boundary: discovery can be public without giving the order path to a marketplace.
- The team is testing whether a shop can stay directly reachable.

KEEP OUT OF SCOPE

- online payment integration, delivery logistics, and customer accounts
- large admin dashboards and full federation story
- production security claims and legal overpromises
- pretending the whole repo is the demo

ONE CLEAN 5-PERSON SPLIT

1. customer page, menu, and QR flow
2. order submit path and basic state handling
3. operator view and ready/received state changes
4. registry/discovery story and proof trace
5. pitch, packet, and live demo setup

If the group does not want the registry in the live demo, keep the direct ordering proof and show the registry only as the technical extension.

THE REPO ALREADY HAS REAL LANES. THE JOB IS TO PICK THE RIGHT ONES AND MAKE THEM PRESENTABLE.

Do not invent a fake architecture for the room. Pick a small honest subset from what already exists in P4P, and make that subset work cleanly as a team.

REPO LANES THE GROUP CAN PICK FROM

- **`registry/`**: discovery lane, announce, heartbeat, nearby lookup, failover story.
- **`demo-node/`**: smallest protocol proof node with direct menu and order path.
- **`pilot-node/`**: more real restaurant-shaped node with operator rooms, SQLite state, and optional module lanes.
- **`client/`**: simple public customer flow and menu/order surface.
- **`docs/` + `tests/`**: one-pagers, packet, demo script, proof language, and smoke path.

KEEP ON GITHUB, NOT IN THE ROOM CLAIM

- **`p4p.payment.godpay-mock`** and **`p4p.payment.chaospay-mock`**: internal mock/debug lanes only.
- **`p4p.payment.mobilepay`**: planned external adapter invitation, not a live payment claim.
- **`p4p.order.print`**: planned printer/POS direction, not the first honest demo requirement.
- **`p4p.notify.email`** and **`p4p.notify.sms`**: planned fallback notification directions.
- **`p4p.trust.cvr-basic`**: later trust/CVR direction, not first hackathon scope.

MODULES THE GROUP CAN ACTUALLY USE

PUBLIC CUSTOMER SIDE

- **`p4p.menu.list`**: plain customer menu surface.
- **`p4p.payment.cash`**: pay-at-pickup boundary without pretending P4P is a payment company.
- **`p4p.customer.status`**: read-only order status for the customer after order submission.

OPERATOR SIDE

- **`p4p.kitchen.screen`**: operator queue for incoming and ready orders.
- **`p4p.catalog.editor`**: restaurant-owned menu/catalog truth.
- **`p4p.pickup.board.basic`**: simple local pickup counter board.

CHOOSE ONE BUILD SHAPE

- Use **`pilot-node/`** if the group wants a more believable operator surface with modules.
- Use **`demo-node/`** if the group wants the smallest technical proof first.
- Use **`client/`** or a very thin custom surface for the customer flow.
- Bring in **`registry/`** only if someone on the team wants the deeper protocol boundary and can keep it explained simply.

MENU · PICKUP · CASH FIRST · OPERATOR QUEUE · STATUS

GOOD OPTIONAL ADD-ONS IF TIME ALLOWS

- **`p4p.menu.photo-map`**: more visual paper-menu style customer surface.
- **`p4p.stock.basic`**: simple stock check before the order proceeds.
- **`p4p.catalog.import.ocr`**: draft OCR/scanned-menu helper beside the catalog editor.
- **`p4p.order.alert.basic`**: local alert/bell self-test lane.
- **`p4p.order.print.backup`**: backup print/spool self-test lane.

ONE CLEAN GITHUB SPLIT

1. **Registry person**: `registry/`, discovery story, technical proof notes.
2. **Node person**: `demo-node/` or `pilot-node/`, menu/order path, module wiring.
3. **Customer person**: `client/`, menu, QR, order submit, status handoff.
4. **Operator person**: kitchen screen, pickup board, catalog/editor polish.
5. **Narrative person**: docs, slides, one-pagers, live demo order.

Practical rule: one owner per lane, one shared merge point, and no mass editing across every folder at once.

IF THE DEMO LANDS, THE NEXT JOB IS NOT MORE HYPE. IT IS A SMALL, CONTROLLED PATH INTO REALITY.

This last page is for the group, judges, and possible supporters. It explains the next path, what help would matter, and how the team can stay honest after the event.

LONGER-TERM PATH

PHASE 1: TIGHTEN THE PACKET

- clean the repo story and public reading path
- keep the non-technical and technical one-pagers aligned
- show one short demo path outsiders can follow fast
- write the fuller whitepaper later as a separate job

PHASE 2: CONTROLLED REAL PILOT

- one real takeaway or food-service partner
- real QR, menu, pickup, operator flow, and local hardware truth
- honest log of what breaks in practice

PHASE 3: MODULE MATURATION

- printer / kitchen ticket lane
- local node / local box lane
- better stock, alert, and pickup-board reliability
- later payment adapters, trust, and certification lanes if the pilot is real

WHAT HELP WOULD ACTUALLY MATTER

- one real pilot partner willing to test a direct route outside marketplace capture
- small follow-up funding or incubation for controlled pilot work
- legal and regulatory guidance around ordering, receipts, and consumer rules
- multilingual or cross-country testing support
- one public-interest or civic-tech partner that cares about local digital resilience

WHAT WE MUST NOT DO AFTER THE EVENT

- do not claim P4P is already a finished restaurant product
- do not imply the payment lane is solved if it is still cash-first
- do not present mock modules as real production partners
- do not describe the whole repo as one coherent finished stack if it is still a proof system
- do not treat hackathon applause as pilot evidence

GROUP DEBRIEF

- [ ] Did people understand the problem quickly?
- [ ] Did they understand why this is not just another food app?
- [ ] Did the direct-order boundary feel meaningful?
- [ ] Did anyone ask for the technical version?
- [ ] Did anyone offer a real pilot path?
- [ ] Did the room push us toward too much scope?
- [ ] Which modules sounded real and useful to outsiders?
- [ ] What should be cut or clarified before the next presentation?

THE SENTENCE WE END ON

This team does not need to prove the whole future of food ordering. It only needs to prove that a shop can stay directly reachable, and that the surrounding software can stay modular instead of collapsing back into one platform owner.

DIRECT ROUTE · MODULAR · CASH FIRST · SMALL PILOT · HONEST SCOPE